

SIZMA

Yazılım Dokümantasyonu

Hacker temalı, Türkçe 10 parmak yazma oyunu
Kurulum yok, derleme yok, backend yok — saf HTML / CSS / JS

Aslıhan Erturhan

07.08.2026 · sürüm 21f24c1 (kaydedilmemiş değişikliklerle)

MIT Lisansı

Bu belge **türetilmiş** bir çıktıdır: kaynağı depodaki README.md ve docs/MIMARI.md dosyalarıdır. Elle düzenlemeyin — python tools/dokuman_uret.py ile yeniden üretin.

İçindekiler

Bölüm I — Kullanım Kılavuzu 3

SIZMA	3
Oynanış	3
Modlar	3
Öğrenme tarafı	4
Oyna	5
Geliştirme	5
Erişilebilirlik	6
Klavye	6
Proje yapısı	6
Tasarım kararları	7
Lisans	8
Durum	8

Bölüm II — Mimari ve Geliştirici Notları 9

SIZMA — Mimari ve Geliştirici Notları	9
1. Temel yaklaşım	9
2. Modül haritası	9
3. Oyun döngüsü ve durum	10
4. Giriş modeli — ön ek eşleştirme	11
5. Satır üretimi	11
6. Veri modeli (`localStorage`)	12
7. Denge	13
8. Eğitim modu (`js/lessons.js`)	13
9. Tarifler	14
10. Test etme	15
11. Bilerek yapılmayanlar	16

Bölüm I — Kullanım Kılavuzu

SIZMA

Hacker temalı, Türkçe 10 parmak yazma oyunu. Kurulum yok, derleme yok, backend yok — saf HTML/CSS/JS.

Terminal ekranında yukarıdan gürültü satırları iner. Her satırın içinde işaretlerle ([komut], \$komut\$, (komut), <komut>, {komut}) gizlenmiş bir komut vardır. Görev: komutu gürültüden ayıklamak ve süre bitmeden doğru yazmak.

Zaman senin canın. Doğru komut süre kazandırır, kaçan komut ve tuzaklar süre götürür.

Oynanış

- **Ayıklama** — her satırda komut yok; saf gürültü satırları cezasızdır, asıl beceri işareti fark etmek.
- **Harf harf doğrulama** — doğru harfler yeşil yanar, yazdığın satıra kilitlenirsin.
- **Yanlış harfin cezası yoktur** — sadece ilerlemez. Amaç panik değil, doğru refleks.
- **Seri (combo)** — art arda komut tamamladıkça puan çarpanı büyür. 3'te seri göstergesi belirir (çarpanın devreye girdiği yer), 5'te terminal **Overdrive**'a geçer: yazılar temanın vurgu rengine, arka plan lacivere döner ve ekran kenarı mavi parlar. Seriyi bozan şeyler: tuzağa düşmek, komut kaçırmak, bomba patlaması/kaçması — **yanlış harf bozmaz**. Overdrive boss sırasında kapanır ve Pratik / Parmak / Eğitim modlarında hiç çıkmaz (skorun ödülü olduğu için).

Düşmanlar

	Tür	Davranış
	Tuzak	İşaret kırmızı yanıp söner — yazarsan ceza, bırakırsan cezasız
	Değişken	Hedef değilken komut ara ara değişir
⌂	Bölünen	Ekranın ortasında ikiye ayrılır
	Saatli bomba	Kendi geri sayımı var; kaçarsa büyük ceza, yazarsan 2x puan
	Boss	"Güvenlik Duvarı" — can barı, uzun komutlar, yoğun tuzak

Boss komutlarının bir uzunluk tavanı vardır (BALANCE.bossMaxLen). Tavan keyfi değil: satır ekranda ne kadar kalıyorsa, komutu hedef hızda yazmak ondan kısa sürmelidir — yoksa kusursuz oynayan biri için bile yetiştirilmesi imkânsız bir komut üretilmiş olur.

Düşmanlar oyun ilerledikçe kademeli açılır.

Modlar

Mod	Ne yapar
Sızma	Asıl oyun: geri sayım, cezalar, düşmanlar, boss.
Pratik	Süre yok, ceza yok, düşman yok. Saat yukarı sayar, sonunda analiz raporu.

Mod	Ne yapar
Günlük	Günün tarihinden tohumlanmış sabit görev — herkes aynı komutları, aynı tuzakları görür.
Parmak	Komut yerine üretilmiş alıştırma dizileri (fff j j j f j f). Ev/üst/alt sıra, sol/sağ el veya "zayıf nokta".
Eğitim	Sıfırdan başlayanlar için 10 derslik müfredat: sıralı kilit, ders başına doğruluk + hız eşiği, yıldızlar.

Eğitim modu

Parmak modu serbest antrenmandır; Eğitim modu ise bir **öğrenme yoludur** — "şimdi ne çalışmalıyım, hazır mıyım" sorusunu oyun cevaplar.

#	Ders	Doğruluk	Hız
1	Dayanak tuşları	%88	12 WPM
2	Ev sırası	%90	14
3	Üst sıra — işaret & orta	%91	14
4	Üst sıra tamamı	%92	16
5	Alt sıra — işaret & orta	%92	16
6	Alt sıra tamamı	%93	18
7	Tüm klavye	%94	20
8	Gerçek komutlar	%94	22
9	Türkçe harfler (ğüşöç)	%95	22
10	Rakam & noktalama	%94	18

- Bir ders **10 alıştırma satırı** sürer; süre baskısı ve ceza yoktur.
- Geçmek için **hem doğruluk hem hız** eşiği tutmalıdır — hızlı ama hatalı yazmak yıldız kazandırmaz.
- Yıldızlar: geçme ★, hedefin 1.2 katı ★★, 1.5 katı ★★★. **Yıldız düşmez** — kötü bir tekrar oynayıp önceki sonucu bozmaz.
- Kilitli dersler **yine de tıklanabilir**: kilit yol gösterir, duvar örmez.
- Dersler harf değil **konum** tabanlıdır; klavye düzenini değiştirmek ilerlemeyi bozmaz. (EN düzeninde Türkçe harfler dersi gizlenir.)

Öğrenme tarafı

Bu bir oyun kadar bir **öğretme aracı**:

- Ekran klavyesi** TR-Q / TR-F / EN düzenlerini destekler, sıradaki tuşu ve hangi parmakla basılacağını gösterir.
- Uyarlanır hedefleme** — 150 tuştan sonra oyun, hata oranının yüksek olan harfleri ve en zayıf parmağının harflerini içeren komutları daha sık gönderir.
- Müfredat** — Eğitim modu sıfırdan başlayanı dayanak tuşlarından gerçek komutlara götürür.
- Analiz** — WPM trend grafiği, klavye ısı haritası, parmak ve sıra bazlı doğruluk, en yavaş / en hatalı harfler.

- **Seviye / XP** — XP doğrudan doğru yazılan karakter sayısıdır. Zorluk seçimiyle veya comboyla oynanamaz.
- **Rozetler** (5 yıldızlı kademeler) ve yerel en iyi 10 maç tablosu.

Tüm veri tarayıcının `localStorage`'ında durur; hesap yok, sunucuya hiçbir şey gitmez.

Verini taşımak / yedeklemek

Analiz panelindeki **dışa aktar** bütün ilerlemeni (istatistik, geçmiş, lider tablosu, rozetler, ders ilerlemesi) tek bir `sizma-data.json` dosyasına indirir; **içe aktar** aynı dosyayı geri yükler.

Dikkat: `localStorage` adres başına ayrılır — `http://127.0.0.1:5500` ile `http://localhost:5500` ya da farklı bir port tarayıcı için ayrı sitelerdir ve ilerlemen "kaybolmuş" gibi görünür. Oyunu hep aynı adresten aç; VS Code Live Server kullanıyorsan portu sabitle.

Tarayıcı kaydetmeye hiç izin vermiyorsa (site verileri engellenmişse ya da sayfa `file://` altında açıldıysa) oyun yine tam çalışır — bellek içi bir depolamaya düşer ve menüde bunu söyleyen bir uyarı gösterir. Tek fark, ilerlemenin sekme kapanınca uçmasıdır.

Oyna

→ aslihan05.github.io/sizma-typing-game

Kurulum gerekmez, tarayıcıda açılır.

Oyun fiziksel klavye için tasarlandı. Dokunmatik bir cihazda başlatmaya çalışırsan bir uyarı çıkar ve üç seçenek sunar:

- **menüye dön** — klavyesiz de istatistiklerine, rozetlerine ve ders ilerlemene bakabilirsin
- **klavyem var, devam et** — klavyeli tablet için
-  **ekrandaki klavyeyle oyna** — oyunun kendi 10 parmak rehberi giriş aygıtına dönüşür; tuşlara dokunarak yazarsın

Seçim hatırlanır. Dokunmatik modda işletim sisteminin klavyesi **açılmaz** — oyun alanı kapanmaz, parmak renkleri ve sıradaki tuş vurgusu görünür kalır ve düzen senin seçtiğin düzendir (TR-Q / TR-F / EN), cihazınkine bağlı değil. Klavyenin altında  **sil** ve **iptal** tuşları belirir.

Yine de: dokunmatik mod **10 parmak öğrenmek için uygun değildir**, tek parmakla dokunarak yazmak kas hafızası kurmaz. Oyunun asıl amacı için gerçek bir klavye kullan.

Telefon ve tabletlerde **istatistikler, rozetler, ders listesi, ayarlar ve veri dışı/içe aktarma** sorunsuz çalışır — ilerlemene her yerden bakabilirsin.

Uygulama olarak kurmak

Oyun bir **PWA**'dır: Chrome / Edge adres çubuğundaki "Yükle" simgesine tıklayınca bilgisayarına kendi ikonunu ve penceresi olan bir uygulama olarak kurulur ve **çevrimdışı** çalışır. Android'de "Ana ekrana ekle" aynı işi görür.

Tüm dosyalar ilk ziyarette önbelleğe alınır (`sw.js`); sonraki açılışlarda internet gerekmez. Veriler zaten `localStorage`'da, sunucuya hiçbir şey gitmez.

Geliştirme

Depoyu indirip `index.html`'i açman yeterli. Ancak service worker ve panoya kopyalama gibi bazı tarayıcı API'leri `file://` altında çalışmaz, bu yüzden küçük bir yerel sunucu önerilir:

```
python -m http.server 8000
```

Sonra tarayıcıda `http://localhost:8000` adresine git.

Service worker localhost / 127.0.0.1 üzerinde **önce ağı** dener, yani düzenlediğin dosya yenilemede olduğu gibi gelir — SURUM artırmana gerek yok. Ağ yoksa yine önbeleğe düşer, dolayısıyla çevrimdışı davranışı geliştirirken de sinayabilirsin (sunucuyu durdurup sayfayı yenile). Gerçek alan adından servis edilen sürüm eskisi gibi önbellek-önceliklidir; **yayına çıkarken sw.js içindeki SURUM sabitini artır**, yoksa kullanıcı güncellemeyi görmez.

PDF dökümantasyonu

docs/SIZMA-Dokümantasyon.pdf, bu README ile docs/MIMARI.md (docs/MIMARI.md)'den **türetilir** — elle düzenlenmez. Döküman değişince yeniden üretin:

```
python tools/dokuman_uret.py
```

Oyunun hâlâ hiçbir bağımlılığı yok; aşağıdakiler yalnızca bu araca ait ve oyunu çalıştırmak için gerekmez: pip install reportlab markdown matplotlib (matplotlib sadece DejaVu fontları için — Türkçe karakterler ReportLab'in gömülü fontunda yok.)

Erişilebilirlik

prefers-reduced-motion açıksa animasyonlar, matrix yağmuru ve sarsma/flash efektleri devre dışı kalır; işlevsel renkler korunur. Ayarlardan tema (Hacker / Synthwave / Cyberpunk / Karanlık), yazı boyutu ve tam ekran seçilebilir.

Ekran okuyucu tarafında: durum satırı aria-live ile duyurulur (oyunun anlatıcısı orası — "tuzak! -3 sn", "ders bitti", depolama uyarısı), yalnızca emoji içeren düğmeler aria-label taşır, ses düğmesi ile seçim grupları (mod, tema, yazı boyutu, ekran klavyesi) aria-pressed ile hangi seçeneğin seçili olduğunu bildirir.

Dürüst sınır: **oyunun kendisi** ekran okuyucuyla oynanamaz. İnen yazıları gözle yakalayıp süreye karşı yazmak buna elverişli değil ve bunu değiştirmeye çalışmak oyunu başka bir şeye dönüştürür. Yapılan iş, oyun dışındaki her şeyin — menü, ayarlar, analiz paneli, ders listesi — gezilebilir olması.

Klavye

Tuş	İşlev
Esc	Kilidi bırak · duraklat/devam
Backspace	Son harfi sil
P	Duraklat

Proje yapısı

index.html	PWA künyesi (ad, ikonlar, renkler)
manifest.json	service worker – çevrimdışı önbellek
sw.js	uygulama ikonları
icons/	görsel tema, CRT efekti, tema değişkenleri
css/style.css	depolama güvenlik ağı – en başta yüklenir
js/storage.js	ekran klavyesi, düzenler, parmak renkleri, ısı haritası
js/keyboard.js	analiz, seviye/XP, grafik, lider tablosu, dışa/içe aktarma
js/stats.js	parmak antrenmanı dizileri
js/drills.js	eğitim modu müfredatı, ders ilerlemesi ve değerlendirme
js/lessons.js	zorluk ve denge sabitleri
js/balance.js	Web Audio ile sentetik terminal sesleri
js/audio.js	rozet/başarım sistemi
js/badges.js	tema, yazı boyutu, tam ekran
js/settings.js	nesne + fiil çarpımından komut üretimi
js/uretici.js	oyun döngüsü, üretim, modlar, ekranlar
js/main.js	

docs/MIMARI.md mimari ve geliştirici notları
docs/SIZMA-Dokümantasyon.pdf türetilmiş belge (bkz. PDF dökümantasyonu)
tools/dokuman_uret.py o belgeyi üreten betik

Derleme adımı ve bağımlılık yok. Tek dış kaynak: Google Fonts üzerinden JetBrains Mono.

Kodun içine girecekler için mimari, veri modeli ve "yeni komut/rozet/ders nasıl eklenir" tarifleri:
docs/MIMARI.md (docs/MIMARI.md).

Tasarım kararları

Birkaç seçim bilinçli ve oyunun tamamını şekillendiriyor:

- **Her satırda komut yok.** Saf gürültü satırları cezasızdır. Denge açısından zorunlu: her satırda komut olsaydı orta zorlukta dakikada ~55 komut inerdi, kimse o hızda yazamaz. Oyun açısından da zorunlu — "ayıklama" görevinin gerçek olması için bazen ayıklanacak bir şey olmamalı.
- **Yanlış harfin cezası yok.** Ceza panik ve tereddüt üretir; yazma öğrenirken istenen şey doğru refleks. Yanlış harf sadece ilerletmez.
- **Zaman = can.** Ayrı can barı yok. Tek kaynak olması hem anlatıyı hem dengeyi sadeleştiriyor: her ödül ve ceza aynı para biriminden ödeniyor.
- **Baskısız modlarda skor hiç gösterilmez.** Pratik, Parmak ve Eğitim'de ceza ve süre yok; skor orada yalnızca "ne kadar oynadığını" ölçerdi. Hem oyun sırasında başlıkta hem sonuç ekranında gizlenir — ders ortasında öğrenciye anlamsız bir sayı göstermemek için. Saat kalır: yukarı sayar ve kendi hızını görmen içindir.
- **XP doğrudan doğru yazılan karakter sayısı.** Ayrı bir puan sayacı tutulmuyor, yani zorluk seçimiyle veya comboyla şişirilemez. "Verileri sıfırla" dendiğinde seviye de dürüstçe sıfırlanır.
- **Derste geçme kriteri hem doğruluk hem hız.** Yalnız doğruluk olsaydı tek parmakla bakarak yazan biri de müfredatı bitirirdi; yalnız hız olsaydı yanlış alışkanlık ödüllendirilirdi. Doğruluk birincil, hız ikincildir.
- **Parmak antrenmanında gerçek kelime kullanılmıyor.** Kelime yazmak belirli bir parmağı izole edemez — "kilidi aç" yazarken sekiz parmak birden çalışır. Parmak kası ancak o parmağın harfleri arka arkaya tekrarlanınca oturur. Kurgu bozulmasın diye diziler işaretlerin içinde "şifre parçası" gibi iner.
- **Komut tekrarı yok.** Komutlar karıştırılmış bir torbadan sırayla çekilir; torba bitmeden hiçbirini tekrar etmez ve torba `localStorage`'da saklandığı için üst üste oynanan oyunlarda da tekrar oluşmaz. Günlük görevde torba dondurulur ki herkes aynı diziye görsün.
- **Komutlar yazılmıyor, üretiliyor.** Sabit liste ne kadar uzasa da torba bitince aynı liste yeniden karışır — döngü kaçınılmazdır, sadece periyodu uzar. Elle komut yazmak *toplamalı* (bir komut yazarsın, +1 olur), kelime listesi *çarpımsal* (bir nesne +5, bir fiil +9 komut). `js/uretici.js` nesne + fiil çarpımından komut kuruyor; elle yazılmış banka yerine değil **yanına** ekleniyor, çünkü oradakiler seçilmiş ve kurguya oturmuş komutlar. Havuz 130 → 2208 (TR). Nesneler belirtme hâliyle yazılı ("çekirdeği"): ünlü uyumu düzenli ama istisnalı, kural yazmak yerine çekilmiş hâli saklamak sıfır hata demek.

Bilerek yapılmayanlar

- **Power-up'lar** — oyuncuyu klavyeden koparıp "ne zaman kullansam" kararına iter.
- **Çok oyunculu mod** — backend, eşleştirme ve hile önleme gerektirir; ayrı bir proje.

Kararı değişenler

- **Dokunmatik oynanış.** Uzun süre "yapılmayacak" listesindeydi: 10 parmak öğreten bir oyunu dokunmatikte oynatmak amacı boşa çıkarır. Karşı gerekçe ağır bastı — telefonuyla

açan biri karşısında ölü bir ekran buluyordu. Şimdi ekran klavyesi bir **giriş aygıtına** dönüşebiliyor, ama asıl gerekçe hâlâ geçerli sayılıyor: mod bir uyarının arkasında duruyor, varsayılan değil ve öğrenme yolu olarak sunulmuyor (bkz. Oyna).

Lisans

MIT (LICENSE) — © 2026 Aslıhan Erturhan

Durum

Oynanabilir ve özellik olarak tamam: beş mod, düşman kadrosu ve boss, analiz, seviye, rozetler, müfredat.

Sıradaki işler:

- **Denge doğrulaması** — `js/balance.js` sayıları ve Eğitim modu ders eşikleri masabaşı hesaplarla belirlendi; gerçek oynanışla sınanıp revize edilecek.
- Zorluk bazlı ayrı istatistikler.
- Hikâye katmanı.

Bölüm II — Mimari ve Geliştirici Notları

SIZMA — Mimari ve Geliştirici Notları

Bu belge kodun içine girecekler içindir: modüllerin sorumlulukları, oyunun durum modeli, veri şeması ve sık yapılan değişikliklerin tarifleri. Oyunun ne olduğu ve nasıl oynandığı için README (../README.md)'ye bakın.

1. Temel yaklaşım

- **Derleme yok, bağımlılık yok.** Saf HTML + CSS + JS. Dosyalar `index.html` içinde sırayla `<script>` ile yüklenir; modül sistemi (ES modules, bundler) kullanılmaz. Her dosya global kapsama yazar.
- **DOM tabanlı oyun.** Canvas yok. Her inen satır bir `<div>`, her harf bir `<span>`. Böylece tema/CSS efektleri ve ekran okuyucu davranışı bedava gelir, karşılığında satır sayısı düşük tutulmak zorundadır (bkz. `maxCmds`).
- **Backend yok.** Tüm kalıcı veri `localStorage`'da.
- **Modülerleştirme ayrı bir iş olarak yapılmaz.** Çalışan kodu yeniden yazmak risktir; bunun yerine her yeni özellik kendi dosyasına yazılır, kod doğal olarak bölünür. `js/main.js` bu yüzden hâlâ en büyük dosyadır.

2. Modül haritası

Yükleme sırası önemlidir — aşağıdaki sıra `index.html`'deki sıradır ve bağımlılık yönünü de gösterir (üsttekiler alttakileri bilmez).

Dosya	Sorumluluk	Dışarıya verdiği başlıcalar
<code>js/storage.js</code>	Depolama güvenlik ağı: <code>localStorage</code> kullanılamıyorsa bellek içi taklidini kurar	<code>SIZMA_DEPOLAMA_KALICI</code>
<code>js/keyboard.js</code>	Ekran klavyesi, düzen tabloları, parmak haritaları, ısı haritası çizimi	<code>KBD_LAYOUTS</code> , <code>KBD_FINGER_MAPS</code> , <code>renderKeyboard()</code> , <code>highlightNextKey()</code>
<code>js/stats.js</code>	Session + lifetime analiz, WPM/doğruluk, seviye/XP, trend grafiği, lider tablosu, dışa/içe aktarma	<code>sStats</code> , <code>trackEvent()</code> , <code>wpmOf()</code> , <code>accuracyOf()</code> , <code>levelInfo()</code> , <code>exportData()</code> , <code>importData()</code>
<code>js/drills.js</code>	Parmak antrenmanı dizisi üretimi	<code>DRILL_TARGETS</code> , <code>makeUniqueDrill()</code> , <code>drillRowLetters()</code> , <code>drillGroup()</code>
<code>js/lessons.js</code>	Eğitim modu müfredatı, ders ilerlemesi, değerlendirme	<code>LESSONS</code> , <code>makeUniqueLessonLine()</code> , <code>evaluateLesson()</code> , <code>recordLessonResult()</code>
<code>js/balance.js</code>	Denge sabitleri	<code>BALANCE</code> , <code>DIFFICULTIES</code>
<code>js/audio.js</code>	Web Audio ile sentetik sesler	<code>playKey()</code> , <code>playComplete()</code> , <code>playGameOver()</code> ...

Dosya	Sorumluluk	Dışarıya verdiği başlıcalar
js/badges.js	Yıldız kademeli rozetler (5 kademe, STAR_COUNT)	BADGES_DB, loadBadges(), evaluateBadges(), renderBadgesPanel()
js/settings.js	Tema, yazı boyutu, tam ekran	(kendi kendine bağlanır)
js/uretici.js	Nesne + fiil çarpımından komut üretimi (TR/EN, normal/boss)	URETILEN_TR, URETILEN_BOSS_TR, URETILEN_EN, URETILEN_BOSS_EN
js/main.js	Oyun döngüsü, satır üretimi, giriş modeli, modlar, tüm ekranlar	startGame(), endGame(), showMenu()

js/lessons.js, js/drills.js'in yardımcılarını (drillGroup, drillRowLetters, isDrillLetter) yeniden kullanır — **drills.js'ten sonra yüklenmelidir.**

js/storage.js **en başta** yüklenmelidir. Sonraki dosyaların bir kısmı üst seviyede localStorage okur (settings.js'te tema/yazı boyutu, main.js'te mod/öğretici/klavye düzeni). Depolama engelliyse bu okumalar istisna fırlatır ve betiği orada keser: menü açılır ama ekran klavyesi çizilmez ve startGame SecurityError atar — yani oyun başlatılamaz. Ağ en başta kurulunca sonraki dosyalar ham localStorage'a hiç dokunmamış olur.

3. Oyun döngüsü ve durum

startGame(zorlukAnahtarı) → requestAnimationFrame(gameLoop) → endGame().

gameLoop(timestamp) her karede sırayla:

1. running değilse çıkar.
2. dt hesaplar. waitingStart **doğruysa** elapsed **ilerlemez** — oyuncu ilk yazılabilir tuşa basana kadar saat işlemez ve süre erimez. Başlatan tuş tüketilmez, ilk harf de yazıma sayılır.
3. Ders bitiş bayrağını (lessonFinish) kontrol eder.
4. Saati işletir: baskısız modlarda yukarı sayar, Sızma'da geri sayar.
5. Satırları aşağı taşır, ekrandan çıkanları cezalandırır/siler.
6. Gerekirse yeni satır üretir, boss zamanlayıcısını yürütür.

Önemli global durum (js/main.js):

Değişken	Anlamı
running, paused, waitingStart	Döngü durumu
sentences[]	Ekrandaki satırlar ({el, charSpans, y, command, isDecoy, ...})
gameMode	sizma · pratik · gunluk · drill · egitim
difficulty, currentDiffName	Seçili DIFFICULTIES girdisi
elapsed, timeLeft, score, streak	Oyun sayaçları
currentLessonNo, lessonLinesDone, lessonFinish	Eğitim modu durumu

Neden lessonFinish bayrağı var: ders dolduğunda endGame()'i doğrudan completeCommand() içinden çağırmak, o fonksiyonun geri kalanını (DOM temizliği, efektler) yarım bıraktı. Bayrak konur, bitiş bir sonraki karede yapılır.

Mod yüklemleri

isPratik() isDrill() isGunluk() isEgitim()

```
isZamansiz() = isPratik() || isDrill() || isEgitim()
```

`isZamansiz()` "süre baskısı yok" demektir: saat yukarı sayar, ceza yoktur, düşman türü üretilmez, skor/rekor gösterilmez. Yeni bir baskısız mod eklerken tek yapılması gereken bu yükleme dahil etmektir.

4. Giriş modeli — ön ek eşleştirme

Oyun oyuncu adına satır **seçmez**. Basılan harfler bir tamponda birikir; tamponla başlayan tüm komutlar "aday" olur ve yazılan kısımları yeşile döner. Yazdıkça adaylar elenir, tek aday kalınca satır kilitlenir.

Kurallar:

- **Yanlış harf yok sayılır**, tamponu sıfırlamaz. (İlk sürümde sıfırlıyordu ve tek yanlış tuş 10 harflik emeği siliyordu — bir yazma oyununda kabul edilemez.) Hata yine analize kaydedilir ve `rejectKey()` görsel sinyali tetikler.
- `Backspace` son harfi geri alır, `Esc` yazımı iptal eder.
- Aday kalmadığında tampon kendiliğinden sıfırlanır — aksi halde yazdığın satır ekrandan çıkınca klavye "ölü" kalıyordu.

Tek giriş yolu: `handleTypedKey(tus)`

Oyun mantığı girişin nereden geldiğini bilmez. İki kaynak buraya bağlanır:

1. Fiziksel klavye — `document` üzerindeki `keydown`.

2. Ekrandaki klavyeye dokunma — `#keyboardSection` üzerinde devredilmiş tek bir `click` dinleyicisi. Her tuşun `dataset.key`'i vardır (bkz. `js/keyboard.js`), boşluk çubuğu dahil.

İşletim sisteminin yazılım klavyesi **bilerek açılmaz**. Denenen ilk yol görünmez bir `<input>` odaklayıp `input` olaylarını okumaktı; çalışıyordu ama klavye ekranın yarısını kaplayıp oyun alanını kapatıyordu. Oyunun zaten çizdiği 10 parmak rehberini giriş aygıtına çevirmek hem yeri korur, hem parmak renklerini ve sıradaki tuş vurgusunu dokunmatikte de görünür tutar, hem de düzeni cihazın klavyesine değil oyuncunun seçimine bağlar.

Dokunma girişi yalnızca oyuncu açıkça "ekrandaki klavyeyle oyna" dediğinde etkinleşir (`sizmaSoftKeyboard`); aksi halde fare kullanıcısı tuşlara yanlışlıkla tıklayınca yazmış olurdu. Mod açıkken `.keyboard` elemanı `.tappable` sınıfını alır ve klavyenin altında `sil / iptal` düğmeleri belirir.

CSS tuzağı: `.keyboard.tappable.key` kuralı (0,3,0) özgüllükte olduğu için `.key.space-key` (0,2,0) genişliğini eziyor ve boşluk çubuğunu normal tuş boyutuna düşürüyordu. Genişlik kuralı bu yüzden `:not(.space-key)` ile sınırlanır. Tuş genişliği de sabit değil `clamp()` ile ekrana bağlıdır — sabit 27px, 360px'lik telefonlarda en kalabalık sırayı (12 tuş + girinti) taşıyordu.

- `refreshCandidates()` her karede değil, yalnızca satır listesi değiştiğinde (`candidatesDirty`) çalışır. Her karede çalışırken 60 fps x ~90 DOM işlemi yapıyor ve yazarken takılmalara yol açıyordu.

5. Satır üretimi

`generateSentence(forceNormal)` tek karar noktasıdır. Sırası önemlidir:

1. Bölünme çocuğu (`forceNormal`) — tavanı dinler.

2. Komut tavanı — ekranda `difficulty.maxCmds` (boss'ta `+bossCmdBonus`) kadar yazılabilir komut varsa satır zorunlu olarak saf gürültü olur. *İş yükünü sabit tutan asıl mekanizma budur*; gürültü oranı tek başına dengeyi kurmaz.

3. Saf gürültü olasılığı (üst üste en fazla 2).

- 4. **Boss** → uzun boss komutu + bossDecoyRate tuzak.
- 5. **Eğitim** → lessonCommand() (Ders 8'de gerçek komut bankası).
- 6. **Parmak** → makeUniqueDrill().
- 7. **Pratik** → düz komut, düşman yok.
- 8. **Sızma** → düşman türleri, enemyMult ile ölçeklenen takvimle açılır.

Komut tekrarı yoktur: komutlar karıştırılmış bir torbadan sırayla çekilir (BAG_KEY ile localStorage'da saklanır), torba bitmeden hiçbirini tekrar etmez. Günlük görevde torba dondurulur (freezeBags()) ve Math.random günün tarihinden tohumlanan bir LCG'ye bağlanır (setSeededRandom()) ki herkes aynı diziyi görsün.

6. Veri modeli (localStorage)

Anahtar	Sabit	İçerik
sizmaStats	STATS_KEY	Ömür boyu analiz — XP/seviye buradan türer
sizmaHistory	HISTORY_KEY	Son 20 oyun (WPM trend grafiği)
sizmaLeaderboard	LEADERBOARD_KEY	En iyi 10 maç
sizmaBadges	BADGES_KEY	Rozet ilerlemesi {id: {v, stars}}
sizmaLessons	LESSONS_KEY	{acilan, sonuclar: {no: {wpm, acc, yildiz}}}
sizmaCmdBags	BAG_KEY	Komut torbaları (tekrar önleme)
sizmaBest	BEST_KEY	En iyi skor
sizmaTutorialSeen	TUTORIAL_KEY	Öğretici gösterildi mi
sizmaMute	AUDIO_MUTE_KEY	Ses kapalı mı
sizmaKeyboardOK	KEYBOARD_OK_KEY	Dokunmatik uyarısı geçildi mi
sizmaSoftKeyboard	SOFT_KEY	Ekran klavyesiyle oynanıyor mu
sizmaLayout, sizmaTheme, sizmaFontSize, sizmaMode, sizmaDrillTarget	—	Tercihler

Dışa/içe aktarma (exportData() / importData()) şunları taşır: stats, history, leaderboard, badges, lessons. Tercihler ve komut torbaları taşınmaz — cihaza özgüdürler.

importData() iki ayrı şeyi birden yapar, ikisini karıştırmayın:

- **Eksik alana toleranslıdır.** Eski bir yedekte lessons yoksa mevcut ders ilerlemesi korunur, hata verilmez.
- **Yanlış şekle toleranslı DEĞİLDİR.** Her alan tek tek doğrulanır (history/leaderboard → Array.isArray, stats/badges/lessons → düz nesne). Şekli tutmayan alan **yazılmaz**, tutan alanlar yüklenir. Hiçbir alan tanınmazsa false döner ve arayüz "dosya hatalı" der.

Yeni bir alan eklerken doğrulama satırını yazmayı atlamayın. Bu koruma kozmetik değil: dosya kullanıcıdan geliyor (yarım inmiş, elle düzenlenmiş ya da yanlışlıkla seçilmiş başka bir .json) ve doğrulama yokken geçerli ama yanlış türde tek bir alan oyunu KALICI olarak kırıyordu — statsEndGame içinde hist.unshift patlıyor, endGame yarıda kesiliyor, sonuç ekranı hiç açılmıyordu. Veri depoda kaldığı için her oyunda tekrar ediyor, tek çıkış yolu "verileri sıfırla" oluyordu.

Aynı kural yükleyiciler için de geçerli: `load*` **fonksiyonları depodaki veri beklenen şekilde değilse boş başlangıca dönmelidir**. `JSON.parse()`'ı `try/catch` içine almak yetmez — geçerli ama yanlış türde bir JSON ("metin", 42) sessizce geçer ve kullanıldığı yerde patlar.

`localStorage` **origin** başına ayrılır (protokol + host + port). Oyunu 127.0.0.1:5500 yerine `localhost:5500` üzerinden açmak, tarayıcı için bambaşka bir sitedir ve veri "kaybolmuş" görünür. Geliştirirken adresi sabit tutun.

İki katmanlı analiz

`js/stats.js` iki nesne tutar: `sStats` (o anki oyun, `statsStartGame()` ile sıfırlanır) ve **lifetime** (`localStorage`'da birikir, `statsEndGame()` ile güncellenir). WPM/doğruluk gibi ölçümler `sStats` üzerinden hesaplanır — Eğitim modunun ders değerlendirmesi de buradan okur.

Bu veri sadece rapor değil, **oyunun girdisidir**: 150 tuştan sonra `refreshWeakLetters()` hata oranı yüksek harfleri ve en zayıf parmağın harflerini seçer, `BALANCE.weakBias` oranında bu harfleri içeren komutlar daha sık gönderilir, hedef tuşlar klavyede işaretlenir.

7. Denge

Bütün sayılar `js/balance.js`'te. Kritik ilişki:

Satır geçiş süresi = oyun alanı yüksekliği / hız
Talep = `maxCmds` / geçiş süresi
Süre ödülü = `timeRewardBase` + `etkinUzunluk` / `difficulty.targetCps`

Kural: bir komutun süre ödülü, o komutu *hedef hızdaki* bir yazıcının yazma süresine eşittir. Böylece hedef hızda yazan oyuncunun saati sabit kalır, daha hızlı olan uzatır, yavaş olan eritir. Yani **"zorluk seviyesi" = "hedef yazma hızı"** demektir (`targetCps`), ki 10 parmak öğretene bir oyun için doğru tanım budur.

Uzun komutlarda azalan getiri vardır: ilk `rewardFullChars` (12) harf tam, sonrası `rewardLongFactor` (0.6) ağırlıkla sayılır. Tavan koymak yerine bu seçildi — tavan, belli bir uzunluktan sonra uzunluk farkını anlamsız kıldı.

`DIFFICULTIES.lineHeight`, CSS'teki `.sentence` yüksekliğinden (18 px) küçük olamaz; yoksa satırlar üst üste biner.

Komut uzunluğunun üst sınırı — `BALANCE.bossMaxLen`

Bir komut, satırın ekranda kaldığı süreden uzun sürede yazılıyorsa **hedef hızda yazan biri için bile yetiştirilmesi imkânsızdır**. Sınır buradan çıkar:

satır ömrü = oyun alanı yüksekliği / (`hız` × `bossSpeedMult`)
yazma süresi = komut uzunluğu / `difficulty.targetCps`
okuma payı = satır ömrü - yazma süresi ← pozitif kalmalı

Orta seviyede: $380 / (24 \times 1.2) = 13.2$ sn satır ömrü. 40 karakteri hedef hızda (3.5 kps) yazmak 11.4 sn, yani okuma payı 1.8 sn. `bossMaxLen` bu yüzden 40; 57 karakterlik bir komut 16.3 sn sürer ve satır ekranda kalmadan biter.

Sınır **hem üretece hem elle yazılmış bankaya** uygulanır (bkz. `bossPool()`).

Ortalamaya değil **medyana** bakın. Üreteç ilk sürümde boss için yalnız sonek kullanıyordu: ortalama 33'ten 35.6'ya çıkmış görünüyordu (zararsız), ama dağılım 35'e sıkışmış ve kısa komut hiç kalmamıştı. Nefes payı tam da kısa komutlardan gelir; medyan 28 → 35 olunca oyun yetiştirmez hâle geldi.

8. Eğitim modu (`js/lessons.js`)

Dersler **harf değil konum** tabanlıdır: "ev sırası" TR-Q'da `asdfg hjklş`, TR-F'de `uieaü tkmlý` demektir. Böylece klavye düzenini değiştirmek ders ilerlemesini bozmaz.

Dayanak (home) tuşları parmak haritasından türetilemez: KBD_FINGER_MAPS TR-Q'da hem h hem j'yi sağ işaret parmağına bağlar ama hangisinin "ev" olduğunu söylemez. Bunun yerine satırdaki **konum** kullanılır — LESSON_ANCHOR_POS = [0,1,2,3,6,7,8,9]. Üç düzende de doğru sonucu verir.

Ders akışı:

```
startLesson(no) → startGame("kolay") // tempo hep kolay; zorluk eşikten gelir
her tamamlanan satır → lessonLinesDone++
lessonLinesDone >= LESSON_LINES (10) → lessonFinish = true
→ gameLoop bir sonraki karede endGame()
→ renderLessonResult() → recordLessonResult(no, wpm, acc)
```

evaluateLesson() saf bir fonksiyondur (kaydetmez), recordLessonResult() kaydeder ve gerekirse bir sonraki dersi açar. Kurallar:

- Geçme = doğruluk **ve** hız eşiği. Yıldız: geçme ★, hedef × 1.2 ★★, × 1.5 ★★★.
- Yarım bırakılan ders (lessonLinesDone < LESSON_LINES) kaydedilmez.
- Kayıt yalnızca iyileştirirse üzerine yazılır; yıldız düşmez.
- Kilit nextVisibleLesson() üzerinden ilerler, yani **gizli dersler zinciri kırmaz**: EN düzeninde Ders 9 (Türkçe harfler) gizlidir ve 8'i geçen oyuncu doğrudan 10'u açar.

9. Tarifler

Yeni komut eklemek

İki yol var ve **ikincisi neredeyse her zaman daha verimli**.

Tek bir komut için: js/main.js içindeki COMMANDS_TR / COMMANDS_EN (boss için BOSS_COMMANDS_*) dizisine ekleyin. Kurguya özel, "lezzetli" komutlar buraya yazılır. Kazanç: +1 komut.

Havuzu büyötmek için: js/uretici.js içindeki U_NESNELER / U_FIILLER listelerine kelime ekleyin. Çarpımsal olduđu için **bir nesne ≈ +5, bir fiil ≈ +9 komut** getirir.

```
{ ad: "belleđi", tur: "veri" }, // nesne – BELİRTME hâliyle
{ ad: "sızdır", turler: ["veri"] }, // fiil – kabul ettiđi türler
```

Üç kural:

- 1. Nesneler belirtme hâliyle yazılır** ("çekirdeđi", "logları"). Ünlü uyumu düzenli ama istisnâlı (çekirdek → çekirdeđi); kural yazmak yerine çekilmiş hâli saklamak sıfır çekim hatası demek.
- 2. Tür şart.** Olmadan "kamerayı çöz", "algoritmayı aç" gibi eşleşmeler üretilir. Türler: ağ · veri · kripto · kilit · sistem · cihaz.
- 3. EN listelerine Türkçe karakter girmez** — o düzende yazılamaz.

Her iki kaynak cmdPool() / bossPool() içinde birleştirilir (bir kez, _havuzlar içinde saklanır) ve torbaya öyle girer. Düzen dili bankayı seçer: EN düzendeyken Türkçe karakterli komut gelmez.

Yeni rozet eklemek

js/badges.js → BADGES_DB'ye bir nesne ekleyin:

```
{
  id: "benzersiz_id", title: "Ad", icon: "□", unit: "birim",
  desc: "Tooltip metni.",
  mode: "life" | "best", // ömür boyu toplam mı, tek oyun rekoru mu
  tiers: [5, 10, 25, 50, 100], // 5 yıldızın eşikleri (STAR_COUNT)
  value: (s, life) => life.birSayac,
}
```

Yeni bir sayaç gerekiyorsa `js/stats.js` → `emptyStats()`'a alanı ekleyin, `statsEndGame()` içindeki `scalars` listesine katın ve oyun içinde `trackEvent("alanAdı")` çağırın.

Yeni ders eklemek

`js/lessons.js` → `LESSONS` dizisine ekleyin. `kind` alanı harf kümesini belirler (dayanak, satır, karışık, komut, türkçe, simge); satır için `row` ve isteğe bağlı `fingers` verin. Yeni bir `kind` gerekiyorsa `lessonLetterSet()` içindeki `switch`'e bir dal ekleyin. Ders numaraları kilit sırasını belirler.

Zorluk ayarlamak

`js/balance.js`. Bir sayıyı tek başına kurcalamadan önce bölüm 7'deki denklemi gözden geçirin — `speed`, `maxCmds` ve `targetCps` birbirine bağlıdır.

Yeni mod eklemek

1. `js/main.js`'te bir `isXxx()` yüklemi tanımlayın; baskısızsa `isZamansiz()`'e katın.
2. `generateSentence()` içine satır üretim dalını ekleyin (sıralamaya dikkat).
3. `index.html`'e `.mode-btn[data-mode="xxx"]` düğmesini ekleyin — dinleyici otomatik bağlanır.
4. `applyModeUI()` içinde menüyü giydirin, `endGame()` içinde sonuç ekranını.

10. Test etme

Otomatik test altyapısı yoktur; doğrulama tarayıcı konsolundan yapılır. Tüm fonksiyonlar global olduğu için doğrudan çağrılabilirler:

```
// Ders harf kümeleri düzene göre doğru mu?
currentLayout = "TR-F"; lessonAnchors().join("") // "uieakmly"
makeLessonLine(3)

// Değerlendirme mantığı
evaluateLesson(1, 40, 80) // hızlı ama hatalı → geçmemeli

// Rozet ilerlemesi
loadBadges()
```

Dikkat: tarayıcı sekmesi görünür değilse `requestAnimationFrame` kare üretmez, yani `running` doğru olsa bile oyun döngüsü ilerlemez. Döngüyü elle çevirmek için `gameLoop(performance.now())` çağırın.

Test ederken `localStorage`'ı kirletmemek için önce **dışa aktar** ile yedek alın; sonra `resetStats()` / `resetLessons()` serbestçe kullanılabilir.

Service worker ve geliştirme

`sw.js` `localhost / 127.0.0.1 / [::1]` üzerinde **ağ-öncelikli** çalışır, yani düzenlenen dosya yenilemede olduğu gibi gelir. Bunun için `fetch`'e `cache: "no-store"` gerekiyor: yerel sunucu `Cache-Control` göndermediği için tarayıcı `Last-Modified`'a bakıp sezgisel önbellekleme yapıyor ve düz `fetch` yine eski dosyayı getiriyordu.

Çevrimdışı davranışı sınamak için sunucuyu durdurup sayfayı yenileyin — ağ başarısız olunca önbelleğe düşülür. Yayına çıkarken `SURUM` sabitini artırın; gerçek alan adında dal önbellek-öncelikli olduğu için artırılmazsa kullanıcı güncellemeyi hiç görmez.

Depolamasız ortamı sınamak

`index.html`'in bir kopyasını alıp `<head>` sonuna şunu ekleyin:

```
<script>
Object.defineProperty(window, "localStorage", {
  configurable: true,
```

```
get() { throw new DOMException("blocked", "SecurityError"); }  
});  
</script>
```

Beklenen: oyun tam çalışır, SIZMA_DEPOLAMA_KALICI === false olur ve menüde uyarı görünür.
js/storage.js betiğini kaldırırsanız klavye çizilmez ve startGame() SecurityError atar — ağın ne işe yaradığını bu fark gösterir.

11. Bilerek yapılmayanlar

- **Power-up'lar** — oyuncuyu klavyeden koparıp "ne zaman kullansam" kararına iter. Zaten süre = kaynak ve beş düşman tipi var; karmaşıklık bütçesi modlara harcanıyor.
- **Çok oyunculu mod** — backend + eşleştirme + hile önleme; ayrı bir proje.
- **Mobil dokunmatik oynanış** — **karar değişti**. Gerekçe ("10 parmak öğreten oyunu dokunmatikte oynatmak amacı boşa çıkarır") hâlâ geçerli sayılıyor, ama telefonuyla açan biri ölü bir ekran buluyordu. Ekran klavyesi artık giriş aygıtı olabiliyor; mod bir uyarının arkasında, varsayılan değil ve öğrenme yolu olarak sunulmuyor.
- **Ayrı bir modülerleştirme fazı** — çalışan 1000+ satırı yeniden yazmak risk; bölünme yeni dosyalar üzerinden doğal olarak gerçekleşir.